



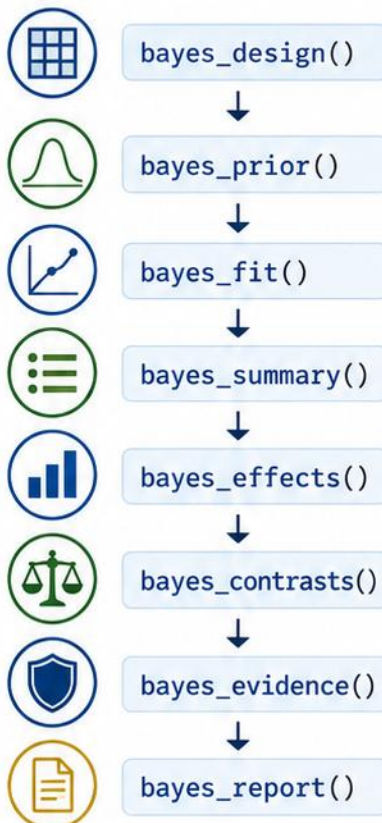
Bayesian Experimental Design Workflows

1 What 0.1.0 Covers

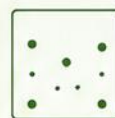
- CRD
- RCBD
- Latin square
- Factorial designs
- Split-plot designs
- Bayes factors
- ROPE
- Effects
- Contrasts
- Diagnostics
- Frequentist crosswalk



2 Core Workflow



3 Key Design Helpers



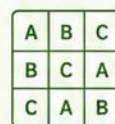
bayes_crd()

Completely Randomized Design



bayes_rcbd()

Randomized Complete Block Design



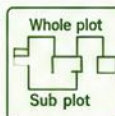
bayes_latin_square()

Latin square design



bayes_factorial()

Factorial designs
(e.g., $A \times B$)



bayes_split_plot()

Split-plot designs
(whole-plot / sub-plot)

4 Computation



bayes_compute()

Manage model computation



bayes_capabilities()

Inspect system capabilities



backend = "cmdstanr"

Default backend



n_cpus

Number of CPU cores to use



chains

Number of MCMC chains



parallel_chains

Run chains in parallel

5 Quick example

```
des <- bayes_rcbd(
  data = agri_rcbd,
  response = yield,
  treatment = cultivar,
  block = block
)
fit <- bayes_fit(
  des,
  engine = "brms",
  backend = "cmdstanr"
)
bayes_summary(fit)
```



Works with **brms** (default) via **cmdstanr**.
See page 2 for priors and models.

6 Teaching Datasets

Built-in example datasets
for each design type.



agri_crd

CRD example dataset



agri_rcbd

RCBD example dataset



agri_latin

Latin square example dataset



agri_factorial

Factorial example dataset



agri_splitplot

Split-plot example dataset



bayesAgriFlow streamlines Bayesian experimental design analysis from design to evidence and reporting.
Designed for agronomy and plant science trials.





Priors, Effects, Contrasts, and Evidence

1 Prior tools



`bayes_prior()`

Specify and inspect priors for model parameters.



`bayes_prior_summary()`

Summarize prior distributions and hyperparameters.



`bayes_prior_predictive()`

Generate prior predictive simulations and plots.



`bayes_prior_sensitivity()`

Assess how priors influence model predictions.

2 Posterior summaries



`bayes_summary()`

Summarize posterior estimates and intervals.



`bayes_effects()`

Estimate main effects and credible intervals.



`bayes_contrasts()`

Compute pairwise contrasts with credible intervals.



`bayes_hypothesis()`

Test directional or point hypotheses (e.g., $>$, $<$, $=$).

3 Evidence layer



`bayes_evidence()`

Compute overall model evidence (marginal likelihood).



`bayes_bf()`

Compute Bayes factors between two models.



`bayes_inclusion_bf()`

Compute inclusion Bayes factors for model terms.

4 ROPE and practical relevance



`bayes_rope()`

Evaluate ROPE results: posterior direction and % within ROPE.



`bayes_equivalence()`

Assess equivalence using credible intervals and ROPE.



`bayes_practical_significance()`

Assess practical significance against user-defined thresholds.

5 Interpretation tips



BF $\neq$ effect size

Bayes factors quantify relative evidence for models, not the magnitude of effects.



ROPE assesses practical relevance

Use ROPE to judge whether effects are meaningfully different from zero.



Contrasts quantify treatment differences

Contrasts answer specific questions with uncertainty quantified.

6 Workflow snapshot

- 1 Define design `bayes_design()`
- 2 Specify priors `bayes_prior()`
- 3 Fit model `bayes_fit()`
- 4 Evaluate relevance `bayes_rope()`
- 5 Quantify evidence `bayes_evidence()`

7 Quick example

```
prior <- bayes_prior(strategy = "regularizing")
fit <- bayes_fit(des, prior = prior)
bayes_rope(fit)
bayes_evidence(fit)
```



Use this sequence to move from principled priors to practical conclusions with quantified evidence.

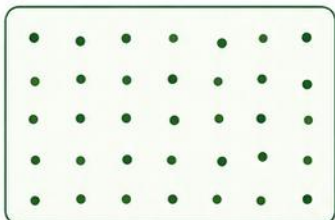




Experimental Designs in Practice

1 CRD

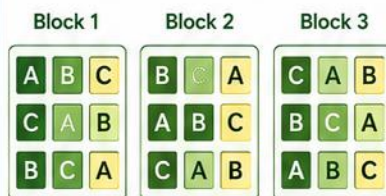
```
bayes_crd()  
bayes_design(type = "crd")
```



- ✓ Completely Randomized Design
- ✓ One treatment factor, no blocks

2 RCBD

```
bayes_rcbd()  
bayes_design(type = "rcbd")
```



- ✓ Randomized Complete Block Design
- ✓ Controls variation among blocks

3 Latin square

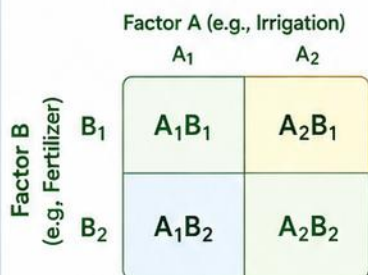
```
bayes_latin_square()  
bayes_design(type = "latin_square")
```

		Columns			
		1	2	3	4
Rows	1	A	B	C	D
	2	B	C	D	A
	3	C	D	A	B
	4	D	A	B	C

- ✓ Controls row and column variation
- ✓ Each treatment appears once per row and once per column

4 Factorial designs

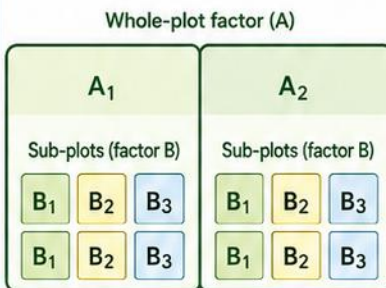
```
bayes_factorial()
```



- ✓ All combinations of factors
- ✓ Estimates main effects and A x B interaction

5 Split-plot designs

```
bayes_split_plot()
```



- ✓ Two-stage randomization
- ✓ Larger units for whole plots, smaller units for sub-plots

6 Design audit

```
bayes_design_audit()
```

-  Check roles
treatment, block, row,
column, whole_plot,
sub_plot
-  Check structure
design type and layout
are consistent
-  Check balance
replication counts per
treatment are as expected
-  Check missing units
identify empty or absent
experimental units

7 Compact workflow example



1. Design object

```
# RCBD example  
des <- bayes_design(  
  data = agri_rcbd,  
  type = "rcbd",  
  treatment = cultivar,  
  block = block  
)  
bayes_summary(des)
```



2. Fit the model

```
fit <- bayes_fit(  
  des,  
  engine = "brms",  
  backend = "cmdstanr"  
)
```



3. Contrast/evidence

```
ct <- bayes_contrasts(  
  fit,  
  comparisons = list(  
    "cultivar" = "pairwise"  
  )  
)  
bayes_summary(ct)
```



Design right. Analyze right. Learn more.

bayesAgriFlow helps you connect experimental design to Bayesian evidence with confidence.





Computation, Diagnostics, and Reporting

1 Computation controls



bayes_compute()
Set computation controls



bayes_compute_info()
Show current compute info



bayes_capabilities()
Inspect system capabilities

Key parameters

backend	Engine backend (e.g., cmdstanr)
n_cpus	Number of CPU cores to use
chains	Number of MCMC chains
parallel_chains	Run chains in parallel
threads_per_chain	Threads per chain (within chain)

2 Stan / brms route

```
bayes_fit(  
  engine = "brms",  
  backend = "cmdstanr"  
)
```



Routes model fitting through **brms** and **cmdstanr** for fast, flexible, and extensible Bayesian computation.

3 Diagnostics



bayes_diagnose()
Summarize MCMC diagnostics



R-hat
Potential scale reduction



ESS
Effective sample size



Divergences
Hamiltonian divergences



MCSE
Monte Carlo standard error

Aim for $R\text{-hat} \approx 1.00$, high ESS, no divergences, and low MCSE.

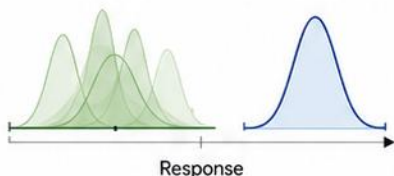
4 Posterior predictive checks



bayes_pp_check()
Posterior predictive checking

Replicated data

Observed data



Compare replicated data distributions to observed data. Look for agreement across key features of the data.

5 Reporting



bayes_report()
Comprehensive report



bayes_report_check()
Report quality checklist

- ✓ Priors
- ✓ Diagnostics
- ✓ Effects
- ✓ Contrasts
- ✓ Evidence
- ✓ Reproducibility

Create transparent, reproducible reports for decision making and publications.

6 Practical checklist

1



Set design
Define the experiment and factors.

2



Choose priors
Use sensible, transparent priors.

3



Fit model
Run **bayes_fit()** with your chosen backend.

4



Inspect diagnostics
Check R-hat, ESS, divergences, MCSE.

5



Report evidence
Use **bayes_report()** to summarize and share.



```
ctrl <- bayes_compute(n_cpus = 8, chains = 4)  
fit <- bayes_fit(des, compute = ctrl)  
bayes_diagnose(fit)  
bayes_report(fit)
```



Adjust controls, refit, and iterate until diagnostics look good and conclusions are stable.



Use these tools together to compute efficiently, check diagnostics carefully, and report Bayesian evidence with confidence.

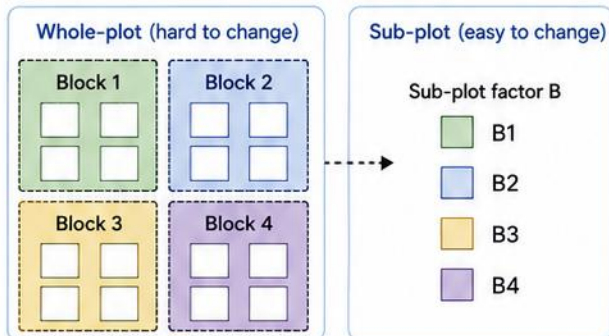




Split-Plot, Hierarchy, and Frequentist Crosswalk

1 Why split-plot matters

Treatments are applied at different scales.
Whole-plot factors are harder to change than sub-plot factors.



`bayes_split_plot()`



Define whole-plot and sub-plot structure and the experimental unit.

2 Hierarchy in the model

The model respects the experimental hierarchy.

Conceptual model (example)			
	Response	yield	
	Fixed effects	$A * B$	Whole-plot factor A, sub-plot factor B
	Random effect	(1block)	Blocks (replication)
	Random effect	(1whole_plot_id)	Whole-plot units nested in block



$\text{yield} \sim A * B + (1 | \text{block}) + (1 | \text{whole_plot_id})$



Random effects capture the correlation within whole plots and blocks.

3 Key outputs

Posterior summaries aligned with the hierarchy.



`bayes_effects()`

Whole-plot effects, sub-plot effects, and interaction



`bayes_contrasts()`

Planned comparisons and contrasts



`bayes_variance_components()`

Variance decomposition by hierarchical level

4 Frequentist crosswalk

Bridge Bayesian results with familiar frequentist outputs.

`bayes_compare_frequentist()`

Frequentist	↔	Bayesian (bayesAgriFlow)
Estimate (e.g., mean)	↔	Posterior mean (or median)
95% CI (approx.)	↔	95% CrI (credible interval)
p-value	↔	Bayes Factor (BF)
ANOVA table	↔	Evidence summary and contrasts

5 Common pitfalls



Pseudoreplication

Do not treat sub-plots as independent whole plots.
Model the hierarchy.



Wrong experimental unit

The experimental unit is the whole plot for whole-plot factors and the sub-plot for sub-plot factors.



Ignoring hierarchy

Omitting $(1 | \text{whole_plot_id})$ understates uncertainty and inflates significance.



Always declare the structure with `bayes_split_plot()` before fitting.

6 Tiny example

Split-plot design with whole-plot factor A (irrigation) and sub-plot factor B (variety).

```
# 1) Declare the split-plot structure
sp <- bayes_split_plot(
  data = agri_data,
  block = "block",
  whole_plot = "A",           # irrigation
  sub_plot = "B",            # variety
  whole_plot_id = "whole_plot_id"
)

# 2) Fit the hierarchical Bayesian model
fit <- bayes_fit(sp, response = "yield")

# 3) Compare with frequentist analysis
cmp <- bayes_compare_frequentist(fit)
cmp
```



bayesAgriFlow helps you design, analyze, and report split-plot experiments with proper hierarchy and clear Bayesian–frequentist alignment.





Quantitative Factors, Regression, and Optima

1 What 0.2.0 Adds

- Quantitative-factor regression
- Polynomial regression
- ANCOVA
- Qualitative × quantitative models
- Biological optimum
- Economic optimum



2 Core new functions



bayes_regression()
Quantitative-factor regression



bayes_polynomial()
Polynomial regression



bayes_ancova()
ANCOVA



bayes_qual_quant()
Qualitative × quantitative models



bayes_optimum()
Biological optimum



bayes_economic_optimum()
Economic optimum

3 Workflow



bayes_design()



bayes_polynomial()



bayes_fit()



bayes_summary()



bayes_optimum()

4 Teaching datasets



agri_dose
Dose-response dataset



agri_ancova
ANCOVA example dataset



agri_qual_quant
Qualitative × quantitative example dataset

5 Quick example

```
des <- bayes_polynomial(  
  data = agri_dose,  
  response = yield,  
  quantitative = nitrogen,  
  degree = 2  
)  
  
fit <- bayes_fit(des)  
  
bayes_optimum(fit)
```



Works with **brms** (default) via **cmdstanr**.
See page 2 for priors and models.

6 Why it matters



Model curved responses

Capture non-linear relationships with polynomial models.



Compare adjusted means

Use ANCOVA and qualitative × quantitative models to compare fairly.



Estimate optimum under uncertainty

Identify biological or economic optima with credible intervals.



bayesAgriFlow extends Bayesian design–analysis workflows to quantitative factors, regression, and decision-focused optima.
Designed for agronomy and plant science trials.

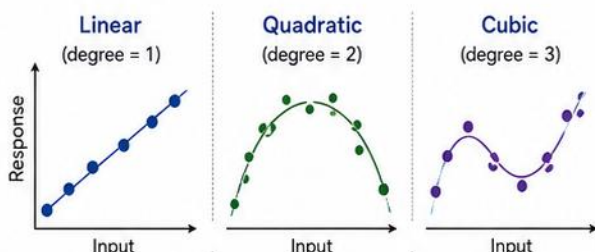




Polynomial Regression and Curved Responses

1 Why polynomial regression?

Many agronomic responses are curved, not perfectly linear.



Higher degrees capture curvature and inflection but may overfit if unnecessary.

2 Main function

bayes_polynomial()

Fits a Bayesian polynomial regression model.

Key parameters:

quantitative

Name of the quantitative input (predictor variable).

degree

Polynomial degree (1, 2, or 3) to control model flexibility.

response

Name of the response variable (outcome).



Uses weakly informative priors by default. See page 2 for priors and models.

3 Degrees supported

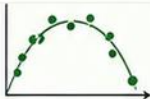
degree = 1 (Linear)

Straight-line trend.



degree = 2 (Quadratic)

Single peak or trough.



degree = 3 (Cubic)

One inflection; more flexible.



4 Key outputs



bayes_summary()



Posterior coefficients, credible intervals.



bayes_predict()



Predicted responses with credible intervals.



bayes_optimum()



Estimated optimum (maximum or minimum).



bayes_compare_models()



Compare degrees (e.g., LOO, WAIC).

5 Teaching example

```
des <- bayes_polynomial(  
  data = agri_dose,  
  response = yield,  
  quantitative = nitrogen,  
  degree = 2  
)  
fit <- bayes_fit(des)  
bayes_predict(fit)
```



Fits a quadratic model and predicts the curved response with uncertainty.

6 Best practices



Keep the domain realistic.

Only predict within the observed input range.



Compare degrees.

Use model comparison to choose the simplest adequate degree.



Inspect uncertainty.

Always visualize credible intervals and residuals.



Do not over-interpret extrapolation.

Predictions far outside the data are uncertain.



bayesAgriFlow extends Bayesian design-analysis workflows to quantitative factors, regression, and decision-focused optima. Designed for agronomy and plant science trials.





Quantitative Models and Posterior Interpretation

1 Model builders



`bayes_regression()`

Linear regression for quantitative predictors.



`bayes_polynomial()`

Polynomial regression for curved relationships.



`bayes_ancova()`

ANCOVA: compare groups while adjusting for quantitative covariates.



`bayes_qual_quant()`

Models with both qualitative (factor) and quantitative predictors.

2 Posterior summaries



`bayes_summary()`

Posterior summaries for parameters and key model terms.



`bayes_effects()`

Estimated marginal effects across the range of a predictor.



`bayes_contrasts()`

Pairwise contrasts and comparisons with credible intervals.



`bayes_predict()`

Posterior predictions and credible intervals for new data.

3 Model checking



`bayes_diagnose()`

Check MCMC convergence and sampling diagnostics.

R-hat,
ESS



`bayes_pp_check()`

Posterior predictive checks to assess model fit.

Posterior
predictive
checks



`bayes_compare_frequentist()`

Compare Bayesian results with traditional (frequentist) fits.

Frequentist
crosswalk

4 Interpretation focus



Slope and trend

Quantify direction and strength of the relationship (linear or nonlinear) with credible intervals.



Adjusted means

Compare groups while accounting for quantitative covariates and confounding factors.



Uncertainty bands

Use posterior intervals to communicate the range of plausible effects and predictions.

5 Short workflow example

```
fit <- bayes_fit(des)
bayes_summary(fit)
bayes_predict(fit)
```



Works with `brms` (default) via `cmdstanr`.
See page 1 for priors and models.

6 Why it matters



Richer insight

Understand how quantitative factors shape outcomes.



Better decisions

Use adjusted estimates and credible intervals for action.



Robust and transparent

Diagnose, validate, and compare models with confidence.



Treat quantitative inputs as numeric predictors, not merely factor labels.



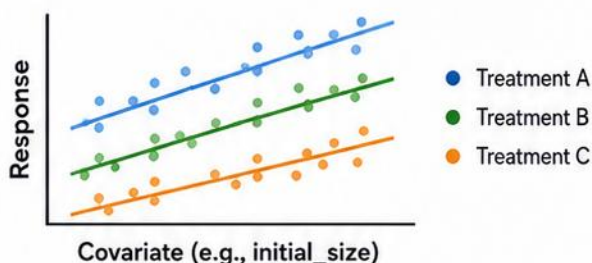


ANCOVA and Qualitative × Quantitative Models

1 ANCOVA

`bayes_ancova()`

Adjust group differences for a continuous covariate.

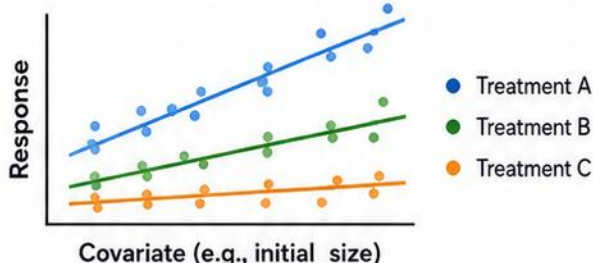


Parallel lines: groups differ in adjusted means;
common slope for the covariate.

2 Qualitative × quantitative

`bayes_qual_quant()`

Allow treatment-specific slopes
(interaction with the covariate).



Non-parallel lines: treatments have
different slopes (interaction).

3 What to estimate



`bayes_effects()`

Adjusted means (least-square means)
for treatments.



`bayes_contrasts()`

Pairwise comparisons and
slope (interaction) differences.



`bayes_predict()`

Fitted lines across the covariate
with uncertainty.

4 When to use



Block unwanted variation by
including a relevant covariate.



Compare treatments at common
covariate values (fair comparisons).



Evaluate treatment-specific
response curves.

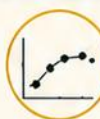
5 Quick example

```
des <- bayes_ancova(  
  data = agri_ancova,  
  response = yield,  
  treatment = cultivar,  
  covariate = initial_size  
)  
fit <- bayes_fit(des)
```



Works with `brms` (default) via `cmdstanr`.
See page 2 for priors and models.

6 ⚠ Check before you conclude



Check linearity of the covariate effect.

Explore residuals vs. covariate and
consider transformations or splines
if needed.



Think carefully about interaction.

If slopes may differ by treatment,
use `bayes_qual_quant()` to avoid
misleading conclusions.



bayesAgriFlow extends Bayesian design–analysis workflows to quantitative factors,
regression, and decision-focused optima.
Designed for agronomy and plant science trials.





Biological and Economic Optima

1 Optimum functions



`bayes_optimum()`

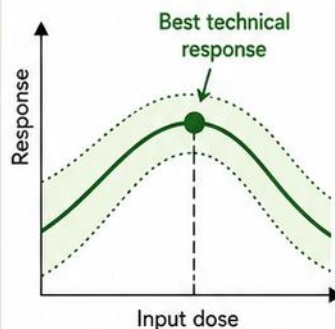
Finds the biological (technical) optimum of the fitted response.



`bayes_economic_optimum()`

Finds the economic optimum that maximizes expected net return.

2 Biological optimum



Posterior optimum
(peak response)

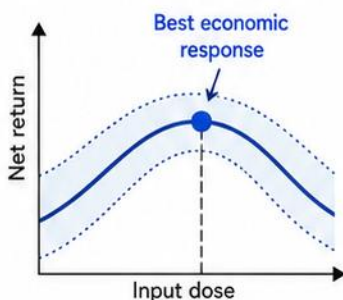


Credible interval
(e.g., 95%)



Optimum within
the studied domain

3 Economic optimum



Price-cost setup

output_price (\$/unit)



input_cost_per_unit



4 What is reported



Optimum estimate

Posterior mean optimum (dose)



Credible interval

e.g., 95% credible interval



Boundary probability

Probability optimum is at/near a boundary



Posterior uncertainty

Spread of the optimum distribution

5 Decision workflow



`bayes_polynomial()`

Specify response
model



`bayes_fit()`

Fit the model and
obtain posterior draws



`bayes_optimum()` /
`bayes_economic_optimum()`

Find biological or
economic optimum

6 Quick example

```
opt1 <- bayes_optimum(  
  fit,  
  domain = range(agri_dose$nitrogen)  
)  
  
opt2 <- bayes_economic_optimum(  
  fit,  
  domain = range(agri_dose$nitrogen),  
  output_price = 1.2,  
  input_cost_per_unit = 0.8  
)
```

7 Tip



Biological and economic optima can differ.

The biological optimum maximizes response, while the economic optimum maximizes net return given prices and costs.



bayesAgriFlow helps quantify and compare biological and economic optima with full posterior uncertainty. Designed for agronomy and plant science trials.





Mixed Models, Repeated Measures, and Advanced Structures

1 What 0.3.0 Adds

- Bayesian mixed models
- Random slopes and hierarchies
- Repeated measures
- Residual correlation structures
- Split-split plots
- Strip-plot experiments
- Non-Gaussian GLMMs



2 Core new functions

-  `bayes_mixed()`
Mixed-effect model builder
-  `bayes_random_slopes()`
Random-slope structures
-  `bayes_repeated()`
Repeated-measures setup
-  `bayes_residuals()`
Residual covariance structures
-  `bayes_split_split()`
Split-split plot design
-  `bayes_strip_plot()`
Strip-plot design
-  `bayes_glmm()`
Non-Gaussian generalized mixed models

3 Workflow



4 Teaching datasets

-  `agri_mixed`
Mixed-model example dataset
-  `agri_repeated`
Repeated-measures example dataset
-  `agri_splitsplit`
Split-split plot dataset
-  `agri_stripplot`
Strip-plot dataset
-  `agri_glmm`
Count / proportion dataset

5 Quick example

```
des <- bayes_mixed(  
  data = agri_mixed,  
  response = yield,  
  fixed = cultivar,  
  random = block  
)  
  
fit <- bayes_fit(des)  
bayes_summary(fit)
```



Works with `brms` (default) via `cmdstanr`.
See page 2 for hierarchical details.

6 Why it matters

-  **Model hierarchical variation**
separate treatment and block effects
-  **Handle dependence**
repeated observations are not independent
-  **Extend beyond Gaussian data**
analyze counts and proportions with uncertainty



bayesAgriFlow 0.3.0 expands the unified Bayesian workflow to mixed models, longitudinal data, advanced designs, and GLMMs.
Designed for agronomy and plant science trials.



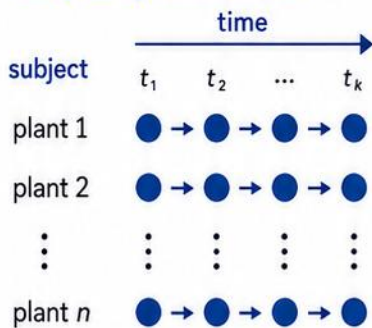


Repeated Measures and Residual Structures

1 Why repeated measures?

Multiple observations on the same unit across time are correlated.

Subject-by-time sketch



Observations within a unit (row) are correlated over time.

2 Main functions



bayes_repeated()
Declare unit and time structure



bayes_residuals()
Specify correlation structure

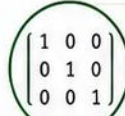


bayes_diagnose()
Inspect convergence and autocorrelation

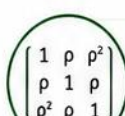


bayes_pp_check()
Posterior predictive checking

3 Residual structures



Independent
No correlation between times.



AR(1)
Correlation decays with lag.



Compound symmetry
Constant correlation between times.



Unstructured
Fully flexible correlation matrix.

4 What to report



bayes_summary()
Posterior means and credible intervals



bayes_effects()
Time profiles and contrasts



bayes_predict()
Predictions for new or existing data



bayes_compare_frequentist()
Frequentist crosswalk for context

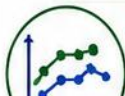
5 Quick example

```
des <- bayes_repeated(  
  data = agri_repeated,  
  response = yield,  
  unit = plant,  
  time = day,  
  treatment = irrigation  
)  
  
fit <- bayes_fit(des)  
  
bayes_predict(fit)
```



Model within-unit dependence instead of treating repeated records as independent.

6 Best practices



Plot trajectories over time
Visualize patterns and heterogeneity.



Compare plausible residual structures
Check sensitivity of inference.



Interpret treatment-by-time patterns with uncertainty
Report credible intervals and contrasts.



Repeated-measures models protect against false certainty by honoring temporal dependence.

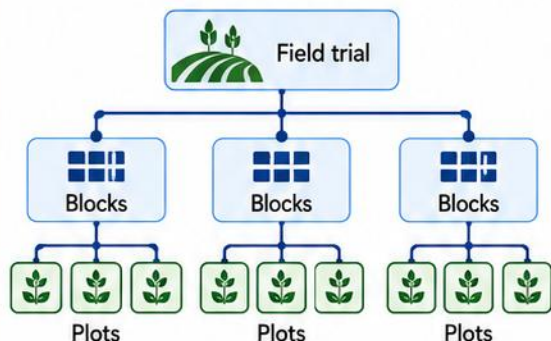




Mixed Models and Random Slopes

1 Why mixed models?

Repeated blocks, plots, sites, and other grouping factors create dependence.



2 Main builders



bayes_mixed()

Declare fixed and random effects



bayes_random_slopes()

Allow group-specific slopes



bayes_variance_components()

Partition variance by hierarchy level



bayes_compare_frequentist()

Crosswalk to familiar mixed-model output

3 Model structure



Response = yield



Fixed effects = cultivar + nitrogen



Random effects =
(1 | block), (1 + nitrogen | site)

```
yield ~ cultivar + nitrogen + (1 | block)
      + (1 + nitrogen | site)
```

4 Key outputs



bayes_summary()

Posterior summaries of fixed and random effects



bayes_effects()

Estimated effects with uncertainty



bayes_contrasts()

Pairwise and custom contrasts



bayes_variance_components()

Variance partitioning by level

5 Quick example

```
des <- bayes_mixed(
  data = agri_mixed,
  response = yield,
  fixed = cultivar + nitrogen,
  random = ~ (1 | block) + (1 + nitrogen | site)
)
fit <- bayes_fit(des)
bayes_variance_components(fit)
```



Estimate treatment effects while accounting for hierarchical variation.

6 Interpretation focus



Shrinkage improves stability

Partial pooling tempers extreme estimates in small groups.



Random slopes reveal heterogeneous response

Effects can vary across sites or other groups.



Variance components show where variability lives

Quantify contributions at each hierarchy level.



Use mixed models when observations are grouped, nested, or partially pooled.





Split-Split Plots and Strip-Plot Experiments

1 Advanced design builders



`bayes_split_split()`
Split-split plot design

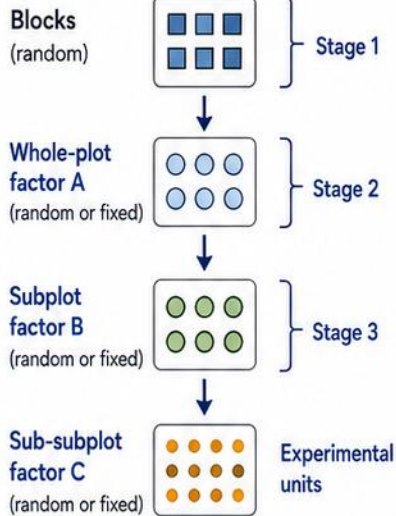


`bayes_strip_plot()`
Strip-plot design

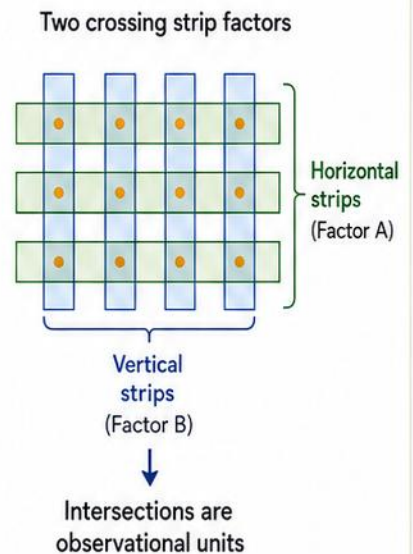


`bayes_design_audit()`
Check structural roles
and balance

2 Split-split plot logic



3 Strip-plot logic



4 What the model tracks



Whole-plot variation
Variation among whole-plot
levels (e.g., irrigation).



Subplot variation
Variation among subplot levels
within whole plots (e.g., cultivar).



**Sub-subplot or intersection
variation**
Variation at the lowest level
(e.g., nitrogen or strip intersections).



**Treatment contrasts
at the correct level**
Estimates respect the design
hierarchy for valid inference.

5 Quick example

```
des <- bayes_split_split(  
  data = agri_splitsplit,  
  response = yield,  
  whole_plot = irrigation,  
  subplot = cultivar,  
  subsubplot = nitrogen,  
  block = block  
)
```

```
fit <- bayes_fit(des)
```

```
bayes_summary(fit)
```



Works with `brms` (default) via `cmdstanr`.
See page 2 for hierarchical details.

6 Common pitfalls



Wrong experimental unit
Analyzing at the wrong level
leads to invalid standard
errors and conclusions.



Ignoring the hierarchy
Treating lower-level factors
as independent inflates
precision.



**Overstating precision
when levels are confused**
Confusing levels understates
uncertainty and overstates
evidence.



Complex experimental designs need hierarchy-aware Bayesian models.





Non-Gaussian GLMMs

1 Why GLMMs?

Counts, proportions, and positive skewed responses are not always Gaussian.



Count outcomes
0, 1, 2, ...



Proportion outcomes
0 to 1



Positive continuous outcomes
Right-skewed



2 Main functions



bayes_glmm()
Generalized mixed-model builder



bayes_family()
Choose the response family



bayes_link()
Specify the link function



bayes_compare_frequentist()
Bayesian vs. ANOVA / GLMM crosswalk

3 Families supported



Binomial
proportions / success counts



Poisson
counts



Negative binomial
overdispersed counts



Gamma
positive continuous outcomes

4 Key outputs



bayes_summary()
Posterior summaries of fixed and random effects



bayes_effects()
Effect sizes and credible intervals



bayes_predict()
Predictions and credible intervals



bayes_pp_check()
Posterior predictive checks

5 Quick example

```
des <- bayes_glmm(  
  data = agri_glmm,  
  response = insects,  
  fixed = cultivar,  
  random = block,  
  family = "poisson"  
)  
  
fit <- bayes_fit(des)  
  
bayes_predict(fit)
```



Works with [brms](#) (default) via [cmdstanr](#).
See page 2 for hierarchical details.

6 Interpretation checklist



Interpret estimates
on the link and response scales



Check overdispersion
and model fit



Report uncertainty,
not only significance



Bayesian GLMMs extend the unified workflow to counts, proportions, and other non-Gaussian traits.





Nonlinear Models, Splines, Bootstrap, and PyMC Extensions

1 What 0.4.0 Adds

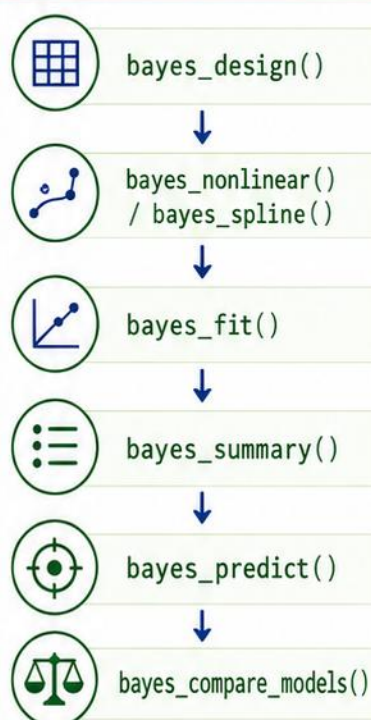
- Nonlinear agronomic response models
- Splines for flexible smooth effects
- Gaussian-process regression
- Bayesian bootstrap
- Flexible nonparametric inference
- Optional PyMC backend
- PyMC-BART for complex prediction



2 Core new functions

-  `bayes_nonlinear()`
Nonlinear response-model builder
-  `bayes_spline()`
Smooth spline-based regression
-  `bayes_gp()`
Gaussian-process models
-  `bayes_bootstrap()`
Bayesian bootstrap workflow
-  `bayes_nonparametric()`
Flexible distribution-free inference
-  `bayes_pymc()`
Optional PyMC backend
-  `bayes_pymc_bart()`
BART-style Bayesian prediction

3 Workflow



4 Teaching datasets

-  `agri_nonlinear`
Nonlinear dose-response dataset
-  `agri_spline`
Smooth-response dataset
-  `agri_bootstrap`
Resampling example dataset
-  `agri_gp`
Spatially smooth response dataset
-  `agri_pymc`
Python-backend example dataset

5 Quick example

```
des <- bayes_nonlinear(  
  data = agri_nonlinear,  
  response = yield,  
  predictor = nitrogen,  
  model = "quadratic_plateau"  
)  
  
fit <- bayes_fit(des)  
  
bayes_predict(fit)
```



Version 0.4.0 opens the door to more flexible response shapes and optional Python-backed engines.

6 Why it matters

-  **Fit realistic curves**
many agronomic responses are not linear
-  **Go beyond rigid formulas**
splines and GPs learn smooth patterns
-  **Explore richer engines**
PyMC and BART support complex prediction



bayesAgriFlow 0.4.0 extends the unified Bayesian workflow to nonlinear, nonparametric, and optional Python-backed modeling.

Designed for agronomy and plant science trials.

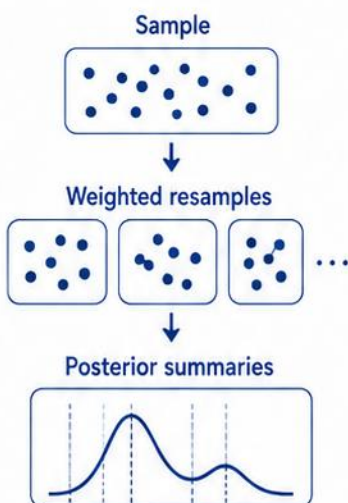




Bayesian Bootstrap and Flexible Nonparametric Inference

1 Why bootstrap?

Some questions need robust uncertainty assessment with minimal distributional assumptions.



2 Main functions



bayes_bootstrap()
Bayesian bootstrap workflow



bayes_nonparametric()
Distribution-free summaries and comparisons



bayes_quantiles()
Posterior quantile summaries



bayes_rank()
Rank probabilities and ordering

3 Typical targets



Median and quantiles



Group differences



Rank probabilities



Distribution overlap

4 Key outputs



bayes_summary()
Posterior means, intervals, and diagnostics



bayes_quantiles()
Quantile estimates and credible intervals



bayes_rank()
Rank probabilities and ordering summaries



bayes_contrasts()
Pairwise contrasts and effect sizes

5 Quick example

```
des <- bayes_bootstrap(  
  data = agri_bootstrap,  
  response = yield,  
  group = cultivar  
)  
  
fit <- bayes_fit(des)  
  
bayes_rank(fit)
```



Works with **brms** (default) via **cmdstanr** and supports custom statistics. See page 2 for hierarchical details.

6 Why it matters



Robust to strong parametric assumptions



Useful for skewed or irregular data



Communicate uncertainty in ordering and quantiles



Bayesian bootstrap extends the unified workflow to flexible, assumption-light inference.

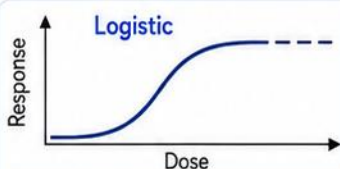
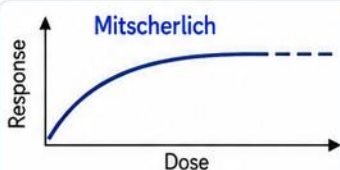
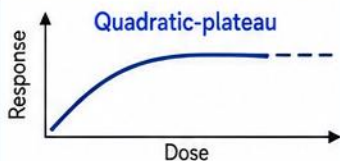




Nonlinear Agronomic Response Models

1 Why nonlinear models?

Many agronomic responses saturate or level off.



2 Main functions



bayes_nonlinear()

Build nonlinear response models



bayes_optimum()

Estimate technical optima



bayes_compare_models()

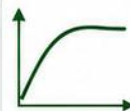
Compare candidate curve families



bayes_predict()

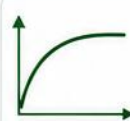
Predict responses with uncertainty

3 Model families



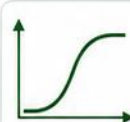
Quadratic-plateau

Rises then plateaus.



Mitscherlich

Rapid early gains, gradual leveling.



Logistic

S-shaped with symmetric acceleration.



Exponential

Continuously accelerating increase.

4 What to report



bayes_summary()

Posterior summaries of parameters and fit.



bayes_optimum()

Estimated optimum and confidence/credible interval.



bayes_predict()

Predicted responses with credible intervals.



bayes_pp_check()

Posterior predictive checks of model fit.

5 Quick example

```
des <- bayes_nonlinear(  
  data = agri_nonlinear,  
  response = yield,  
  predictor = nitrogen,  
  model = "mitscherlich"  
)  
  
fit <- bayes_fit(des)  
  
bayes_optimum(fit)
```



Change `model` = to try different curve families.

6 Best practices



Compare biologically plausible curve families

Let the data guide which shape fits best.



Avoid extrapolating far beyond the observed range

Focus conclusions within the data-supported region.



Report posterior uncertainty around the optimum

Quantify uncertainty in optimum location and response.

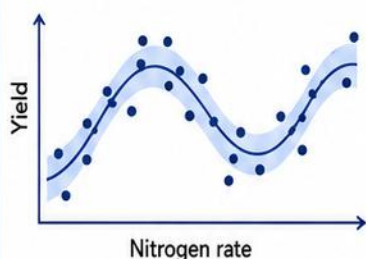




Splines and Gaussian-Process Regression

1 Why flexible smoothers?

Responses in agronomy and ecology are often smooth but not easily described by a fixed formula.



Flexible smoothers adapt to the data while controlling overfitting.

2 Main functions



`bayes_spline()`

Smooth spline-based regression



`bayes_gp()`

Gaussian-process regression



`bayes_predict()`

Smooth predictions with intervals



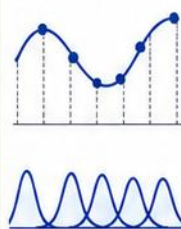
`bayes_compare_models()`

Compare spline, GP, and parametric models

3 Spline vs. GP

Splines

basis functions
+ smooth
penalty



Gaussian process

covariance-
driven
smoothness



Both yield smooth fits with quantified uncertainty; GP smoothness comes from the covariance structure.

4 Key outputs



`bayes_summary()`

Model fit metrics and uncertainty summary



`bayes_effects()`

Estimated smooth effect and credible band



`bayes_predict()`

Predictions with credible intervals



`bayes_derivative()`

Local slope / rate of change along the curve

5 Quick example

```
des <- bayes_spline(  
  data = agri_spline,  
  response = yield,  
  predictor = nitrogen,  
  knots = 5  
)  
  
fit <- bayes_fit(des)  
  
bayes_predict(fit)
```



Choose knots to balance flexibility and parsimony; 5–10 is a common starting range.

6 Interpretation focus



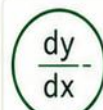
Capture smooth nonlinearity

Let the data reveal the shape.



Inspect credible bands along the curve

Assess uncertainty where it matters.



Use derivatives to identify increasing or leveling regions

Understand local slope and change.



Splines and Gaussian processes provide flexible Bayesian smoothing without forcing a rigid equation.





PyMC and PyMC-BART Integration

1 Why optional Python backends?

Some advanced models and BART-style prediction can benefit from PyMC-based engines.



bayesAgriFlow keeps the workflow unified while letting you leverage powerful Python ecosystems when needed.

2 Main functions



`bayes_pymc()`

Route the model through PyMC



`bayes_pymc_bart()`

BART-style Bayesian prediction



`bayes_backend_status()`

Inspect backend availability



`bayes_predict()`

Posterior prediction from PyMC fits

3 When to use



Flexible nonlinear prediction



Complex interactions



Ensemble-style function learning



Optional alternative backend

4 Workflow



`bayes_design()`



`bayes_pymc()`
`/ bayes_pymc_bart()`



`bayes_fit()`



`bayes_summary()`



`bayes_predict()`

5 Quick example

```
des <- bayes_pymc_bart(  
  data = agri_pymc,  
  response = yield,  
  predictors = c(nitrogen,  
                 rainfall, cultivar)  
)  
fit <- bayes_fit(des)  
bayes_predict(fit)
```



Requires a working Python installation with PyMC (and optional BART dependencies).

6 Reporting checklist



State the backend used



Report predictive uncertainty and diagnostics



Compare with simpler brms-based alternatives when helpful



PyMC integration keeps the API unified while expanding access to richer Bayesian engines.





Spatial, G×E, Multivariate, and Advanced Outcomes

1 What 0.5.0 Adds

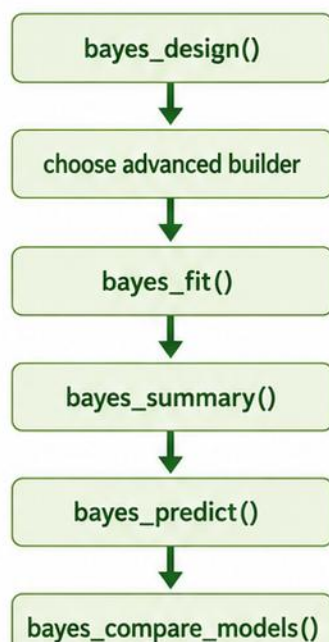
- Spatial field trials
- Genotype-by-environment modeling
- Multivariate responses
- Measurement error and missing data
- Censored and ordinal outcomes
- Finite mixtures
- Distributional regression



2 Core new functions

-  **bayes_spatial()** – Spatial field-trial builder
-  **bayes_gxe()** – Genotype-by-environment models
-  **bayes_multivariate()** – Joint-response models
-  **bayes_missing()** – Missing-data handling
-  **bayes_measurement_error()** – Error-in-variables modeling
-  **bayes_censored()** – Censored/truncated outcomes
-  **bayes_ordinal()** – Ordinal outcome models
-  **bayes_mixture()** – Finite-mixture models
-  **bayes_distributional()** – Model mean and variance together

3 Workflow



4 Teaching datasets

-  **agri_spatial** – Field-trial grid dataset
-  **agri_gxe** – Multi-environment trial dataset
-  **agri_multi** – Multivariate trait dataset
-  **agri_missing** – Dataset with missingness
-  **agri_outcomes** – Censored / ordinal examples

5 Quick example

```
des <- bayes_gxe(data = agri_gxe,
                 response = yield,
                 genotype = cultivar,
                 environment = site)

fit <- bayes_fit(des)

bayes_summary(fit)
```



Version 0.5.0 extends the unified workflow to richer agronomic data structures.

6 Why it matters

-  Respect field dependence
-  Learn across environments
-  Model traits jointly
-  Handle imperfect data with uncertainty



bayesAgriFlow 0.5.0 broadens Bayesian agronomic analysis to spatial, multivariate, and complex outcome settings.



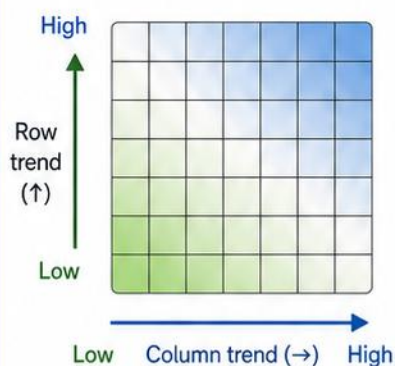


Spatial Field Trials



1 Why spatial models?

Neighboring plots can be correlated, and field trends can bias treatment comparisons.



2 Main functions



bayes_spatial() –
Spatial field-trial builder



bayes_rowcol() –
Row-column trend
structures



bayes_field_map() –
Plot layout declaration



bayes_predict() –
Spatial predictions
with uncertainty

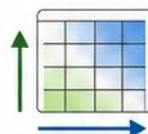


bayes_pp_check() –
Posterior predictive
checks

3 What the model can include



Treatment effects



Row and
column trends



Local spatial
dependence



Adjusted
treatment means

4 Key outputs



bayes_summary() –
Posterior treatment
summaries



bayes_effects() –
Treatment contrasts



bayes_predict() –
Spatial surface
predictions



bayes_residuals() –
Residual spatial
diagnostics

5 Quick example

```
des <- bayes_spatial(  
  data = agri_spatial,  
  response = yield,  
  treatment = cultivar,  
  row = row,  
  col = col)
```

```
fit <- bayes_fit(des)  
bayes_predict(fit)
```



Use spatial models when
plot location influences
the response.

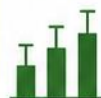
6 Best practices



Map the field layout



Inspect residual
spatial pattern



Report adjusted
treatment uncertainty



Compare spatial vs
nonspatial fits

**Spatial Bayesian models protect inference
when field heterogeneity is not random.**



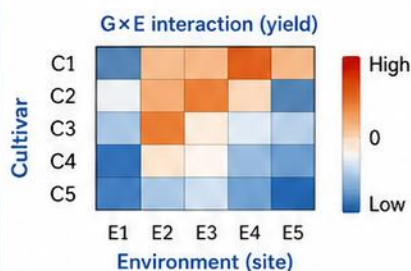


Genotype-by-Environment and Multivariate Responses

1 Why joint modeling?

Cultivars respond differently across environments due to G×E interaction.

Multiple traits are often correlated, so modeling them together yields more accurate inference and better decisions.



Multiple traits (example)



4 Key outputs

bayes_summary() – Posterior means and intervals

bayes_effects() – Environment-specific contrasts

bayes_rank() – Ranking with uncertainty

bayes_correlations() – Trait-association summaries

2 Main functions

bayes_gxe() – Genotype-by-environment models

bayes_multivariate() – Joint-response models

bayes_interactions() – Interaction summaries

bayes_correlations() – Trait correlation estimates

bayes_predict() – Environment- or trait-specific predictions

3 Model components

Genotype main effects
Baseline differences among cultivars.

Environment main effects
Average differences among environments.

G×E interaction
Differential cultivar responses across environments.

Trait covariance
Correlations among multiple traits.

Shared uncertainty
Borrowing information across effects and traits.

5 Quick example

```
des <- bayes_gxe(  
  data = agri_gxe,  
  response = yield,  
  genotype = cultivar,  
  environment = site  
)  
  
fit <- bayes_fit(des)  
  
bayes_effects(fit)
```

Joint models help compare cultivars across diverse conditions.

6 Interpretation focus

Look for crossover interaction

Report environment-specific uncertainty

Use multivariate models for correlated traits

Avoid ranking without interval estimates



G×E and multivariate Bayesian models help agronomy move from isolated means to structured biological interpretation.

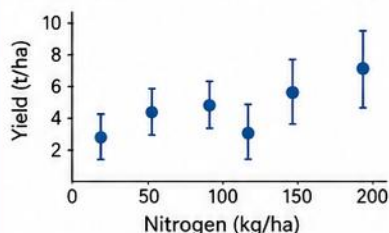


Measurement Error and Missing Data

1 Why model imperfection?

Agronomic datasets often include noisy measurements and missing observations.

Ignoring them can distort inference.



Field	Nitrogen (kg/ha)	Rainfall (mm)	Yield (t/ha)
F1	50	320	4.2
F2	100	—	5.1
F3	150	410	—
F4	200	—	8.3
F5	75	280	3.8

4 Key outputs



bayes_summary()
– Parameter summaries



bayes_impute()
– Imputed values with intervals



bayes_predict()
– Predictions with full uncertainty



bayes_compare_models()
– Compare alternative assumptions

2 Main functions



bayes_measurement_error()
– Error-in-variables builder



bayes_missing()
– Missing-data workflow



bayes_impute()
– Posterior imputation summaries



bayes_sensitivity()
– Sensitivity analysis



bayes_predict()
– Predictions that propagate uncertainty

3 What can be modeled



Response error



Predictor error



Missing responses



Missing covariates



Joint uncertainty propagation

5 Quick example

```
des <- bayes_missing(
  data = agri_missing,
  response = yield,
  predictors = c(nitrogen,
                 rainfall)
)

fit <- bayes_fit(des)

bayes_impute(fit)
```



Imputation should reflect uncertainty, not create false certainty.

6 Best practices



Describe the missing-data mechanism



Do sensitivity analysis



Separate measurement noise from biology



Report uncertainty of imputed quantities



Bayesian models can integrate missingness and measurement error directly into agronomic inference.

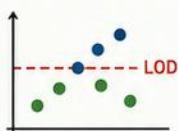




Censored, Ordinal, Mixture, and Distributional Models

1 Why advanced outcome models?

Some responses are censored, ordinal, heterogeneous, or require modeling more than the mean.



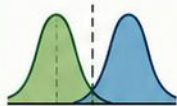
Censoring threshold

Values are only known above/below a limit.



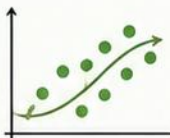
Ordered categories

Outcomes have a natural order.



Two-component mixture

Data arise from subpopulations.



Mean-variance relationship

Variability changes with the mean.

2 Main functions

b() `bayes_censored()` – Censored or truncated outcomes

b() `bayes_ordinal()` – Ordered categorical models

b() `bayes_mixture()` – Finite-mixture models

b() `bayes_distributional()` – Model mean and variance together

b() `bayes_compare_models()` – Compare plausible outcome structures

3 When to use



Detection limits

Responses are censored by instruments or assays.



Disease severity scores

Outcomes are ordered categories.



Latent subpopulations

Data come from heterogeneous groups.



Heteroskedastic responses

Variability changes with predictors or level.

4 Key outputs



bayes_summary() – Posterior parameter summaries



bayes_predict() – Outcome probabilities or predicted distributions



bayes_effects() – Contrasts on the appropriate scale



bayes_pp_check() – Assess fit to complex outcomes

5 Quick example

```
des <- bayes_ordinal(  
  data = agri_outcomes,  
  response = severity,  
  predictors =  
    cultivar + fungicide  
)
```

```
fit <- bayes_fit(des)  
bayes_predict(fit)
```



Choose the model family to match how the data are generated.

6 Interpretation checklist



Respect the response scale



Check component meaning in mixture models



Report probabilities and uncertainty



Model dispersion when variability changes



Advanced outcome models help Bayesian agronomy align inference with how responses are actually observed.





1 What 0.6.0 Adds

- Model uncertainty and predictive averaging
- Projection-predictive selection
- Decision analysis with utility and regret
- Value of information
- Research-decision support



2 Core new functions



`bayes_model_average()`
– Predictive averaging across models



`bayes_projpred()`
– Projection-predictive selection



`bayes_decision()`
– Utility and regret analysis

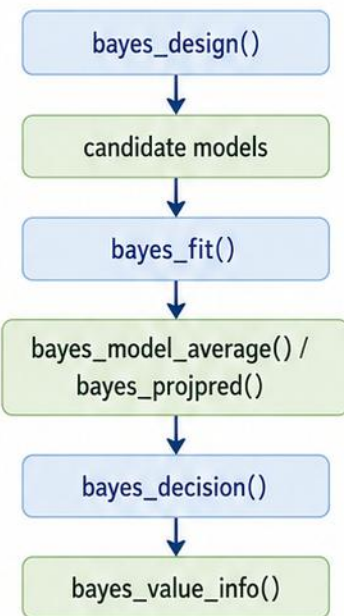


`bayes_value_info()`
– Value-of-information analysis



`bayes_research_decision()`
– Support follow-up studies

3 Workflow



4 Teaching datasets



`agri_selection`
– Variable-selection dataset



`agri_decision`
– Utility-based choice dataset



`agri_valueinfo`
– Research-design dataset



`agri_modelavg`
– Competing-model dataset

5 Quick example

```
fit <- bayes_fit(des)
avg <- bayes_model_average(fit_list)
bayes_summary(avg)
```



Version 0.6.0 turns Bayesian inference into a framework for choosing and acting.

6 Why it matters



Acknowledge uncertainty across models



Prefer predictive relevance



Connect inference to decisions



Quantify whether more data are worth collecting

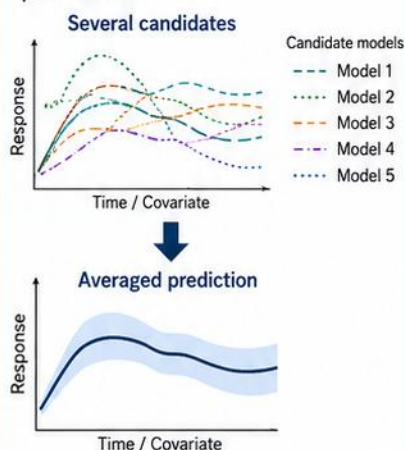




Model Uncertainty and Predictive Averaging

1 Why average models?

Multiple plausible models may explain the data nearly equally well. Averaging can account for model uncertainty and stabilize prediction.



2 Main functions



bayes_model_average()
– Combine models predictively



bayes_weights()
– Posterior or stacking weights



bayes_compare_models()
– Compare candidate models



bayes_predict()
– Averaged predictions



bayes_summary()
– Averaged parameter and predictive summaries

3 Typical ingredients



Candidate models



Predictive weights



Stacking / averaging



Model-averaged uncertainty

4 Key outputs



Model weights

How much each model contributes.



Averaged predictions

Combined prediction and intervals.



Expected predictive performance

Cross-validated or out-of-sample scores.



Sensitivity to model set

How results change with different candidates.

5 Quick example

```
mavg <- bayes_model_average(  
  models = fit_list,  
  method = 'stacking')
```

```
bayes_weights(mavg)
```

```
bayes_predict(mavg)
```



Averaging is often safer than declaring one model the winner.

6 Interpretation focus



Inspect weight concentration

Are weights diffuse or dominated by one model?



Compare predictive accuracy

Use out-of-sample performance.



Report averaged intervals

Reflect total model uncertainty.



Use biologically plausible candidate sets
Ensure models are defensible.



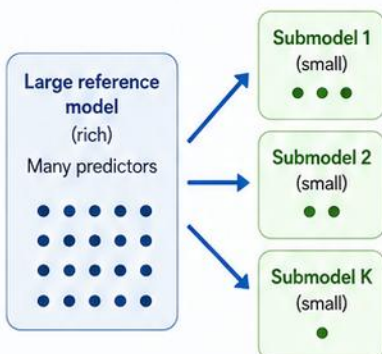
Model averaging acknowledges uncertainty in structure instead of pretending a single model is certainly correct.



Projection-Predictive Selection

1 Why projection predictive?

Simpler models are often preferred when they preserve predictive performance.



Project the rich model onto simpler submodels.

2 Main functions



bayes_projpred() –
Projection-predictive selection



bayes_reference_model() –
Define the rich reference model



bayes_select_variables() –
Rank candidate predictors



bayes_compare_models() –
Compare submodels



bayes_predict() –
Evaluate predictive adequacy

3 Selection logic



Reference model
Rich model with many candidate predictors



Projection to submodels
Project the reference model onto simpler submodels



Predictive loss
Evaluate predictive loss for each submodel



Parsimonious final model
Choose the simplest model with acceptable loss

4 Key outputs



Variable ranking
Importance of candidate predictors



Submodel path
Sequence of nested submodels



Predictive performance curve
Predictive loss vs. number of predictors



Recommended compact model
Smallest model that keeps predictions good enough

5 Quick example

```
sel <- bayes_projpred(  
  reference = fit_full)
```

```
bayes_select_variables(sel)
```

```
bayes_compare_models(sel)
```



Choose the smallest model that keeps the predictions good enough.

6 Best practices



Start with a biologically rich reference model



Evaluate predictive degradation



Prefer interpretable compact models



Report the selected model path



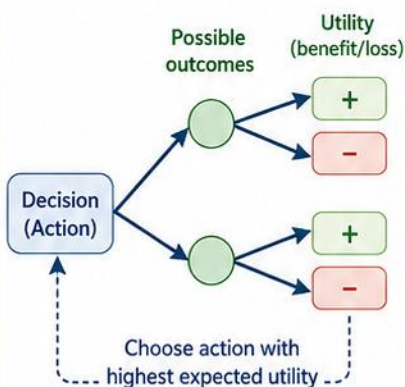


Bayesian Decision Analysis, Utility, and Regret



1 Why decision analysis?

Inference becomes more useful when linked to actions, gains, losses, and consequences.



2 Main functions



bayes_decision() –
Evaluate decisions
under uncertainty



bayes_utility() –
Define utility functions



bayes_regret() –
Summarize regret



bayes_compare_actions() –
Compare candidate
actions



bayes_predict() –
Predict outcomes
feeding decisions

3 Decision ingredients



Actions



Possible outcomes



Utilities / losses



Posterior uncertainty



Expected utility

4 Key outputs



Expected utility



Expected regret



**Optimal action
probabilities**



**Sensitivity to
assumptions**

5 Quick example

```
dec <- bayes_decision(  
  pred = fit,  
  actions = c('low_N',  
              'medium_N',  
              'high_N')  
)  
  
bayes_regret(dec)  
  
bayes_compare_actions(dec)
```



The best action is not always
the one with the largest
mean response.

6 Interpretation checklist



State the utility
function



Check sensitivity to
prices or losses



Report regret as
well as utility



Explain the decision
context clearly



Bayesian decision analysis turns posterior uncertainty
into action-oriented agronomic guidance.



Value of Information and Research Decisions

1 Why value of information?

More data have a cost.
Research should be expanded only when the expected learning is worth it.



Current uncertainty

Potential data collection

Improved decision

Wide uncertainty about outcomes and decisions

Consider a study or experiment (cost required)

Reduced uncertainty and better choices

2 Main functions

- `bayes_value_info()` – Estimate value of information
- `bayes_evpi()` – Expected value of perfect information
- `bayes_evsi()` – Expected value of sample information
- `bayes_research_decision()` – Compare follow-up study options
- `bayes_decision()` – Link value calculations to action choices

3 Core concepts



Current uncertainty
Uncertainty about model inputs and outcomes



Decision consequences
Payoffs of alternative decisions



Perfect information
Knowing the true state of nature



Sample information
Information learned from a study or experiment



Net benefit of more data
Value of improved decisions minus study cost

4 Key outputs



EVPI

Expected value of perfect information



EVSI

Expected value of sample information



Recommended sample size or study option

Best follow-up study based on value



Value vs cost comparison

Compare expected value of information with study cost

5 Quick example

```
voi <- bayes_value_info(
  decision = dec,
  study_options = c('small_trial',
                    'large_trial')
)

bayes_evpi(voi)

bayes_research_decision(voi)
```



Note: Collect more data when the expected gain from learning exceeds the study cost.

6 Best practices



Define realistic study options



Account for research cost



Summarize value and uncertainty



Use VOI to prioritize future experiments



Value-of-information analysis helps decide not only what to conclude, but **whether** more research is worth doing.